

Section 10.4 - Advanced use of RNNs - CPU

December 13, 2025

1 Advanced use of RNNs

- Recurrent dropout
- Stacking recurrent layers
- Bidirectional RNNs

```
[1]: import tensorflow as tf

# ---- Compatibility patch for older TF + newer Keras RNN backend ----
def _install_defun_with_attributes(internal_ns):
    # internal_ns should have a .function submodule
    if hasattr(internal_ns, "function"):
        fn_mod = internal_ns.function
        if not hasattr(fn_mod, "defun_with_attributes"):
            def defun_with_attributes(func=None, **kwargs):
                # Ignore extra attributes; just wrap with tf.function
                return tf.function(func, autograph=kwargs.get("autograph",
↪False))
            fn_mod.defun_with_attributes = defun_with_attributes

# Patch tf.__internal__
if hasattr(tf, "__internal__"):
    _install_defun_with_attributes(tf.__internal__)

# Patch tf.compat.v2.__internal__ (this is the one from your error trace)
if hasattr(tf, "compat") and hasattr(tf.compat, "v2") and hasattr(tf.compat.v2,
↪ "__internal__"):
    _install_defun_with_attributes(tf.compat.v2.__internal__)

# Now import keras from TensorFlow (avoid plain "import keras")
from tensorflow import keras
from tensorflow.keras import layers

print("TF version:", tf.__version__)
print("Keras from tf:", keras.__version__)
```

TF version: 2.13.1

Keras from tf: 2.10.0

1.1 The Jena weather dataset

- Dataset measured at weather station at Max Planck Institute for Biogeochemistry in Jena, Germany
- 14 weather-related quantities: recorded every 10 minutes during 2009-2016
- Create the temperature array as the target of prediction
- Discard the Data-time column, the rest as features

```
[2]: import os

fname = os.path.join("jena_climate_2009_2016.csv")

with open(fname) as f:
    data = f.read()

lines = data.split("\n")
header = lines[0].split(",")
lines = lines[1:]
print(header)
print(len(lines))

['Date Time', 'p (mbar)', 'T (degC)', 'Tpot (K)', 'Tdew (degC)', 'rh (%)', 'VPmax (mbar)', 'VPact (mbar)', 'VPdef (mbar)', 'sh (g/kg)', 'H2OC (mmol/mol)', 'rho (g/m**3)', 'wv (m/s)', 'max. wv (m/s)', 'wd (deg)']
420451
```

1.1.1 Parsing the data

- 420,551 lines of data – each line is a timestep
- Observe yearly periodicity

```
[3]: import numpy as np

temperature = np.zeros((len(lines),))
raw_data = np.zeros((len(lines), len(header) - 1))
for i, line in enumerate(lines):
    values = [float(x) for x in line.split(",")[1:]]
    temperature[i] = values[1]
    raw_data[i, :] = values[:]
```

1.1.2 Data splitting

- First 50% for training, next 25% for validation and last 25% for testing
- Timeseries data: more recent data for validation and testing than for training - predict future, given the past, not the reverse!

```
[4]: num_train_samples = int(0.5 * len(raw_data))
num_val_samples = int(0.25 * len(raw_data))
```

```

num_test_samples = len(raw_data) - num_train_samples - num_val_samples
print("num_train_samples:", num_train_samples)
print("num_val_samples:", num_val_samples)
print("num_test_samples:", num_test_samples)

```

```

num_train_samples: 210225
num_val_samples: 105112
num_test_samples: 105114

```

1.1.3 Normalizing the data

- Each timeseries on a different scale → Normalization each independently
- Mean and standard deviation are computed from the training dataset

```

[5]: mean = raw_data[:num_train_samples].mean(axis=0)
raw_data -= mean
std = raw_data[:num_train_samples].std(axis=0)
raw_data /= std

```

1.1.4 Instantiating datasets for training, validation, and testing

- What does a sample look like?
- Samples in dataset are highly redundant → Generate the samples on the fly
- `timeseries_dataset_from_array()` - given an array of timeseries data, provide windows extracted from the original timeseries

- 1) Sample rate = 6: keep one data point per hour
- 2) Sequence length = 120: back to 5 days
- 3) Delay = sample rate * (sequence length + 24 - 1)
- 4) A dataset – a streaming generator, not a concrete tensor

```

[6]: import numpy as np
from tensorflow import keras

sampling_rate = 6
sequence_length = 120
delay = sampling_rate * (sequence_length + 24 - 1)
batch_size = 256

train_dataset = keras.utils.timeseries_dataset_from_array(
    raw_data[:-delay],
    targets=temperature[delay:],
    sampling_rate=sampling_rate,
    sequence_length=sequence_length,
    shuffle=True,
    batch_size=batch_size,
    start_index=0,

```

```

        end_index=num_train_samples)

val_dataset = keras.utils.timeseries_dataset_from_array(
    raw_data[:-delay],
    targets=temperature[delay:],
    sampling_rate=sampling_rate,
    sequence_length=sequence_length,
    shuffle=True,
    batch_size=batch_size,
    start_index=num_train_samples,
    end_index=num_train_samples + num_val_samples)

test_dataset = keras.utils.timeseries_dataset_from_array(
    raw_data[:-delay],
    targets=temperature[delay:],
    sampling_rate=sampling_rate,
    sequence_length=sequence_length,
    shuffle=True,
    batch_size=batch_size,
    start_index=num_train_samples + num_val_samples)

```

1.2 Recurrent dropout

- Variant of dropout in recurrent layers to fight overfitting
- Randomly zero out input units of a layer to break happenstance correlations
- Input dropout mask applied to input: $\text{input_t} = \text{input_mask} * \text{input_t}$
- Recurrent dropout mask applied to the final hidden state: $\text{h_t} = \text{recurrent_mask} * \text{h_t}$
- Same dropout masks (one for input units and one for recurrent activations) for all timesteps
- Representations = internal vectors computed at each timestep
- Recurrent activations = internal vectors computed at each timestep AFTER nonlinearity
- Each sequence appearance - a pair of dropout masks

```

[7]: # Hide the warning messages
import tensorflow as tf
import logging

tf.get_logger().setLevel(logging.ERROR)

from tensorflow import keras
from tensorflow.keras import layers

inputs = keras.Input(shape=(sequence_length, raw_data.shape[-1]))
x = layers.LSTM(32, recurrent_dropout=0.25)(inputs)
x = layers.Dropout(0.5)(x)
outputs = layers.Dense(1)(x)
model = keras.Model(inputs, outputs)

```

```

callbacks = [
    keras.callbacks.ModelCheckpoint("jena_lstm_dropout_CPU.h5",
                                    save_best_only=True)
]

model.compile(optimizer="rmsprop", loss="mse", metrics=["mae"])

history = model.fit(train_dataset,
                    epochs=30,
                    validation_data=val_dataset,
                    callbacks=callbacks)

model = keras.models.load_model("jena_lstm_dropout_CPU.h5")
print(f"Test MAE: {model.evaluate(test_dataset)[1]:.2f}")

```

```

Epoch 1/30
819/819 [=====] - 165s 199ms/step - loss: 28.3574 -
mae: 3.9252 - val_loss: 9.6191 - val_mae: 2.4048
Epoch 2/30
819/819 [=====] - 160s 195ms/step - loss: 14.7522 -
mae: 2.9853 - val_loss: 9.2975 - val_mae: 2.3603
Epoch 3/30
819/819 [=====] - 156s 191ms/step - loss: 14.0389 -
mae: 2.9091 - val_loss: 9.3116 - val_mae: 2.3651
Epoch 4/30
819/819 [=====] - 157s 192ms/step - loss: 13.5703 -
mae: 2.8606 - val_loss: 9.6754 - val_mae: 2.4055
Epoch 5/30
819/819 [=====] - 157s 191ms/step - loss: 13.1925 -
mae: 2.8205 - val_loss: 9.4561 - val_mae: 2.3869
Epoch 6/30
819/819 [=====] - 156s 191ms/step - loss: 12.8592 -
mae: 2.7831 - val_loss: 9.3741 - val_mae: 2.3746
Epoch 7/30
819/819 [=====] - 156s 190ms/step - loss: 12.5469 -
mae: 2.7474 - val_loss: 9.3362 - val_mae: 2.3689
Epoch 8/30
819/819 [=====] - 157s 192ms/step - loss: 12.2336 -
mae: 2.7148 - val_loss: 9.4744 - val_mae: 2.3907
Epoch 9/30
819/819 [=====] - 156s 191ms/step - loss: 11.9604 -
mae: 2.6824 - val_loss: 9.2777 - val_mae: 2.3622
Epoch 10/30
819/819 [=====] - 156s 191ms/step - loss: 11.7469 -
mae: 2.6579 - val_loss: 9.4251 - val_mae: 2.3918
Epoch 11/30
819/819 [=====] - 156s 191ms/step - loss: 11.3785 -
mae: 2.6186 - val_loss: 9.3256 - val_mae: 2.3814

```

Epoch 12/30
819/819 [=====] - 157s 191ms/step - loss: 11.1564 - mae: 2.5935 - val_loss: 9.5654 - val_mae: 2.4183

Epoch 13/30
819/819 [=====] - 157s 191ms/step - loss: 11.0205 - mae: 2.5764 - val_loss: 9.7868 - val_mae: 2.4331

Epoch 14/30
819/819 [=====] - 157s 191ms/step - loss: 10.8343 - mae: 2.5525 - val_loss: 9.5634 - val_mae: 2.4074

Epoch 15/30
819/819 [=====] - 156s 191ms/step - loss: 10.6253 - mae: 2.5293 - val_loss: 9.9092 - val_mae: 2.4522

Epoch 16/30
819/819 [=====] - 157s 192ms/step - loss: 10.5506 - mae: 2.5184 - val_loss: 9.8445 - val_mae: 2.4333

Epoch 17/30
819/819 [=====] - 156s 191ms/step - loss: 10.3684 - mae: 2.4951 - val_loss: 9.8721 - val_mae: 2.4469

Epoch 18/30
819/819 [=====] - 157s 191ms/step - loss: 10.2750 - mae: 2.4854 - val_loss: 9.8734 - val_mae: 2.4414

Epoch 19/30
819/819 [=====] - 157s 192ms/step - loss: 10.2006 - mae: 2.4755 - val_loss: 10.0408 - val_mae: 2.4705

Epoch 20/30
819/819 [=====] - 156s 191ms/step - loss: 10.0967 - mae: 2.4647 - val_loss: 9.9691 - val_mae: 2.4645

Epoch 21/30
819/819 [=====] - 157s 192ms/step - loss: 10.0008 - mae: 2.4533 - val_loss: 10.1528 - val_mae: 2.4804

Epoch 22/30
819/819 [=====] - 156s 190ms/step - loss: 9.8570 - mae: 2.4321 - val_loss: 10.0092 - val_mae: 2.4621

Epoch 23/30
819/819 [=====] - 156s 191ms/step - loss: 9.8433 - mae: 2.4277 - val_loss: 10.3644 - val_mae: 2.5141

Epoch 24/30
819/819 [=====] - 157s 191ms/step - loss: 9.7835 - mae: 2.4219 - val_loss: 9.9835 - val_mae: 2.4513

Epoch 25/30
819/819 [=====] - 157s 192ms/step - loss: 9.6867 - mae: 2.4097 - val_loss: 10.0687 - val_mae: 2.4811

Epoch 26/30
819/819 [=====] - 156s 191ms/step - loss: 9.6711 - mae: 2.4076 - val_loss: 10.2436 - val_mae: 2.4937

Epoch 27/30
819/819 [=====] - 157s 192ms/step - loss: 9.5691 - mae: 2.3956 - val_loss: 10.4375 - val_mae: 2.5191

```

Epoch 28/30
819/819 [=====] - 156s 191ms/step - loss: 9.4905 - mae:
2.3838 - val_loss: 10.3145 - val_mae: 2.5008
Epoch 29/30
819/819 [=====] - 156s 191ms/step - loss: 9.4344 - mae:
2.3756 - val_loss: 10.6507 - val_mae: 2.5401
Epoch 30/30
819/819 [=====] - 156s 190ms/step - loss: 9.3430 - mae:
2.3671 - val_loss: 10.6515 - val_mae: 2.5391
405/405 [=====] - 12s 30ms/step - loss: 10.6937 - mae:
2.5774
Test MAE: 2.58

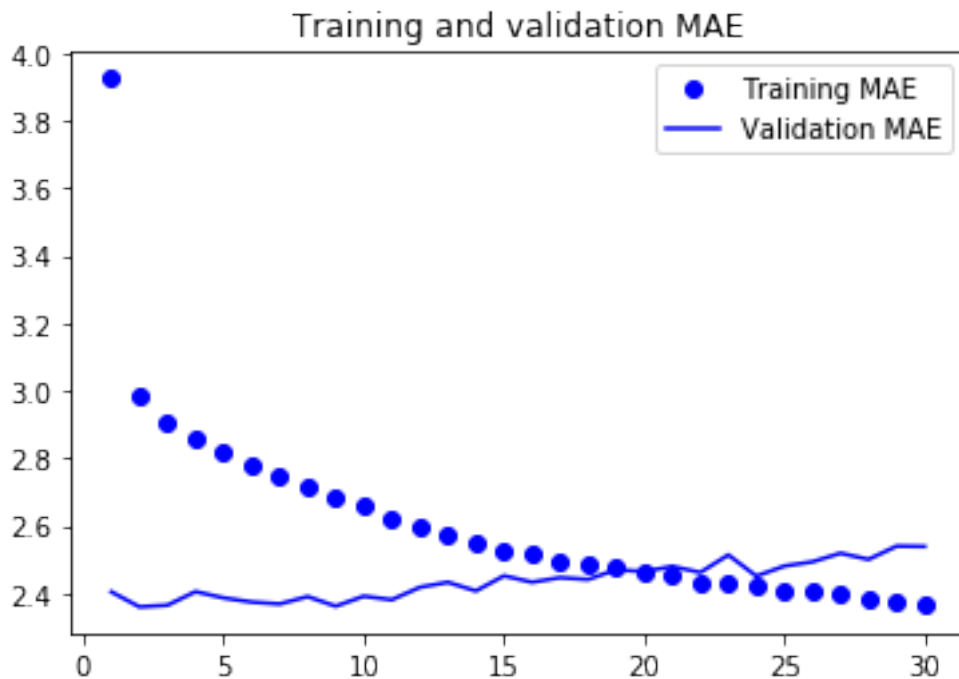
```

```

[8]: import matplotlib.pyplot as plt

loss = history.history["mae"]
val_loss = history.history["val_mae"]
epochs = range(1, len(loss) + 1)
plt.figure()
plt.plot(epochs, loss, "bo", label="Training MAE")
plt.plot(epochs, val_loss, "b", label="Validation MAE")
plt.title("Training and validation MAE")
plt.legend()
plt.show()

```



1.3 Stacking recurrent layers

- Increase the capacity and expressive power of the network
- Intermediate layers return full sequences
- GNMT architecture (Google Translate, ~2016–2020): deep seq2seq RNN with attention; encoder has 8 LSTM layers (1 bidirectional + 7 unidirectional), decoder has 8 unidirectional LSTM layers, with residual connections

```
[9]: inputs = keras.Input(shape=(sequence_length, raw_data.shape[-1]))
x = layers.GRU(32, recurrent_dropout=0.5, return_sequences=True)(inputs)
x = layers.GRU(32, recurrent_dropout=0.5)(x)
x = layers.Dropout(0.5)(x)
outputs = layers.Dense(1)(x)
model = keras.Model(inputs, outputs)

callbacks = [
    keras.callbacks.ModelCheckpoint("jena_stacked_gru_dropout_CPU.h5",
                                    save_best_only=True)
]

model.compile(optimizer="rmsprop", loss="mse", metrics=["mae"])

history = model.fit(train_dataset,
                    epochs=30,
                    validation_data=val_dataset,
                    callbacks=callbacks)

model = keras.models.load_model("jena_stacked_gru_dropout_CPU.h5")

print(f"Test MAE: {model.evaluate(test_dataset)[1]:.2f}")
```

```
Epoch 1/30
819/819 [=====] - 300s 364ms/step - loss: 24.3640 -
mae: 3.6641 - val_loss: 9.4815 - val_mae: 2.3985
Epoch 2/30
819/819 [=====] - 300s 367ms/step - loss: 13.9941 -
mae: 2.9007 - val_loss: 8.6356 - val_mae: 2.2761
Epoch 3/30
819/819 [=====] - 299s 365ms/step - loss: 13.1739 -
mae: 2.8157 - val_loss: 8.9630 - val_mae: 2.3252
Epoch 4/30
819/819 [=====] - 298s 364ms/step - loss: 12.6063 -
mae: 2.7513 - val_loss: 8.6268 - val_mae: 2.2872
Epoch 5/30
819/819 [=====] - 299s 365ms/step - loss: 12.1409 -
mae: 2.7027 - val_loss: 8.7887 - val_mae: 2.3134
Epoch 6/30
819/819 [=====] - 300s 367ms/step - loss: 11.6497 -
mae: 2.6495 - val_loss: 8.7136 - val_mae: 2.2900
```


Epoch 7/30
819/819 [=====] - 300s 367ms/step - loss: 11.2749 - mae: 2.6064 - val_loss: 8.9751 - val_mae: 2.3349

Epoch 8/30
819/819 [=====] - 300s 366ms/step - loss: 10.9418 - mae: 2.5685 - val_loss: 9.0296 - val_mae: 2.3347

Epoch 9/30
819/819 [=====] - 298s 364ms/step - loss: 10.6259 - mae: 2.5308 - val_loss: 9.4952 - val_mae: 2.4091

Epoch 10/30
819/819 [=====] - 298s 363ms/step - loss: 10.3595 - mae: 2.4983 - val_loss: 9.1768 - val_mae: 2.3438

Epoch 11/30
819/819 [=====] - 299s 364ms/step - loss: 10.0577 - mae: 2.4632 - val_loss: 9.7608 - val_mae: 2.4171

Epoch 12/30
819/819 [=====] - 299s 365ms/step - loss: 9.8404 - mae: 2.4364 - val_loss: 9.7834 - val_mae: 2.4271

Epoch 13/30
819/819 [=====] - 300s 366ms/step - loss: 9.5568 - mae: 2.4014 - val_loss: 9.5682 - val_mae: 2.3971

Epoch 14/30
819/819 [=====] - 299s 365ms/step - loss: 9.3936 - mae: 2.3807 - val_loss: 10.1202 - val_mae: 2.4752

Epoch 15/30
819/819 [=====] - 299s 365ms/step - loss: 9.1958 - mae: 2.3569 - val_loss: 10.5488 - val_mae: 2.5257

Epoch 16/30
819/819 [=====] - 297s 363ms/step - loss: 9.0407 - mae: 2.3351 - val_loss: 10.4301 - val_mae: 2.4994

Epoch 17/30
819/819 [=====] - 299s 365ms/step - loss: 8.9085 - mae: 2.3189 - val_loss: 10.7820 - val_mae: 2.5437

Epoch 18/30
819/819 [=====] - 299s 365ms/step - loss: 8.8234 - mae: 2.3085 - val_loss: 10.8768 - val_mae: 2.5454

Epoch 19/30
819/819 [=====] - 300s 367ms/step - loss: 8.6614 - mae: 2.2878 - val_loss: 10.8266 - val_mae: 2.5441

Epoch 20/30
819/819 [=====] - 299s 365ms/step - loss: 8.5258 - mae: 2.2674 - val_loss: 10.9760 - val_mae: 2.5505

Epoch 21/30
819/819 [=====] - 298s 364ms/step - loss: 8.4453 - mae: 2.2603 - val_loss: 10.7382 - val_mae: 2.5194

Epoch 22/30
819/819 [=====] - 299s 366ms/step - loss: 8.3817 - mae: 2.2505 - val_loss: 10.8358 - val_mae: 2.5432

```

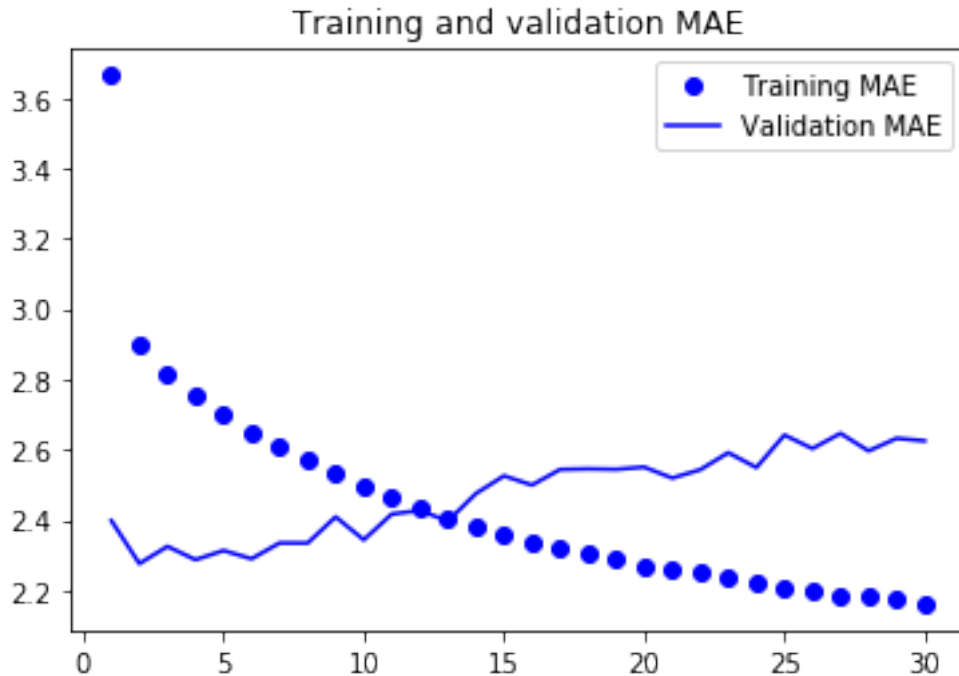
Epoch 23/30
819/819 [=====] - 301s 367ms/step - loss: 8.2851 - mae:
2.2408 - val_loss: 11.3599 - val_mae: 2.5915
Epoch 24/30
819/819 [=====] - 300s 366ms/step - loss: 8.1551 - mae:
2.2222 - val_loss: 11.0235 - val_mae: 2.5492
Epoch 25/30
819/819 [=====] - 298s 364ms/step - loss: 8.0643 - mae:
2.2051 - val_loss: 11.9369 - val_mae: 2.6415
Epoch 26/30
819/819 [=====] - 300s 366ms/step - loss: 8.0003 - mae:
2.2006 - val_loss: 11.5023 - val_mae: 2.6031
Epoch 27/30
819/819 [=====] - 298s 364ms/step - loss: 7.9269 - mae:
2.1876 - val_loss: 11.9973 - val_mae: 2.6463
Epoch 28/30
819/819 [=====] - 300s 366ms/step - loss: 7.8679 - mae:
2.1813 - val_loss: 11.3756 - val_mae: 2.5964
Epoch 29/30
819/819 [=====] - 298s 364ms/step - loss: 7.8209 - mae:
2.1728 - val_loss: 11.6082 - val_mae: 2.6323
Epoch 30/30
819/819 [=====] - 300s 366ms/step - loss: 7.7386 - mae:
2.1619 - val_loss: 11.8218 - val_mae: 2.6250
405/405 [=====] - 22s 54ms/step - loss: 9.4548 - mae:
2.4102
Test MAE: 2.41

```

```

[10]: loss = history.history["mae"]
      val_loss = history.history["val_mae"]
      epochs = range(1, len(loss) + 1)
      plt.figure()
      plt.plot(epochs, loss, "bo", label="Training MAE")
      plt.plot(epochs, val_loss, "b", label="Validation MAE")
      plt.title("Training and validation MAE")
      plt.legend()
      plt.show()

```



1.4 Bidirectional RNNs

- RNNs are order-dependent
- Two layers process the input sequence in two directions and merge their representations
- Catch patterns that may be overlooked by a unidirectional RNNs
- May learn different yet useful representations → worth being exploiting
- Not very useful for temporal timeseries data (forecasting). Why?
- Useful in natural language processing (NLP)

```
[11]: inputs = keras.Input(shape=(sequence_length, raw_data.shape[-1]))
x = layers.Bidirectional(layers.LSTM(16))(inputs)
outputs = layers.Dense(1)(x)

model = keras.Model(inputs, outputs)

callbacks = [
    keras.callbacks.ModelCheckpoint("jena_bidirectional_RNN.h5",
                                    save_best_only=True)
]

model.compile(optimizer="rmsprop", loss="mse", metrics=["mae"])

history = model.fit(train_dataset,
                    epochs=10,
                    validation_data=val_dataset,
```

```

callbacks=callbacks)

model = keras.models.load_model("jena_bidirectional_RNN.h5")
print(f"Test MAE: {model.evaluate(test_dataset)[1]:.2f}")

```

```

Epoch 1/10
819/819 [=====] - 138s 166ms/step - loss: 26.8848 - mae: 3.7259 - val_loss: 10.8911 - val_mae: 2.5668
Epoch 2/10
819/819 [=====] - 135s 165ms/step - loss: 9.4620 - mae: 2.3996 - val_loss: 10.0682 - val_mae: 2.4500
Epoch 3/10
819/819 [=====] - 137s 167ms/step - loss: 8.5775 - mae: 2.2781 - val_loss: 10.1655 - val_mae: 2.4731
Epoch 4/10
819/819 [=====] - 136s 165ms/step - loss: 8.0416 - mae: 2.2031 - val_loss: 10.2522 - val_mae: 2.4922
Epoch 5/10
819/819 [=====] - 136s 166ms/step - loss: 7.6291 - mae: 2.1469 - val_loss: 10.6714 - val_mae: 2.5534
Epoch 6/10
819/819 [=====] - 136s 166ms/step - loss: 7.2824 - mae: 2.0972 - val_loss: 10.9435 - val_mae: 2.5748
Epoch 7/10
819/819 [=====] - 136s 166ms/step - loss: 7.0309 - mae: 2.0634 - val_loss: 11.5322 - val_mae: 2.6374
Epoch 8/10
819/819 [=====] - 136s 166ms/step - loss: 6.8306 - mae: 2.0344 - val_loss: 11.2525 - val_mae: 2.6243
Epoch 9/10
819/819 [=====] - 136s 166ms/step - loss: 6.5848 - mae: 1.9970 - val_loss: 11.3448 - val_mae: 2.6168
Epoch 10/10
819/819 [=====] - 136s 165ms/step - loss: 6.4092 - mae: 1.9702 - val_loss: 11.2889 - val_mae: 2.6195
405/405 [=====] - 12s 28ms/step - loss: 10.6347 - mae: 2.5741
Test MAE: 2.57

```

```

[12]: loss = history.history["mae"]
      val_loss = history.history["val_mae"]
      epochs = range(1, len(loss) + 1)
      plt.figure()
      plt.plot(epochs, loss, "bo", label="Training MAE")
      plt.plot(epochs, val_loss, "b", label="Validation MAE")
      plt.title("Training and validation MAE")
      plt.legend()

```

```
plt.show()
```

